CROSS-IS-SAT

A Cross-Checking Based Polynomial-Time Method for Boolean Satisfiability

ABSTRACT

The Boolean Satisfiability Problem (SAT), which is still essential to computational complexity theory, was the first problem to be shown to be NP-complete. In this paper, we introduce CROSS_IS_SAT, a constructive method that uses systematic cross-checking between clauses to determine the satisfiability of Boolean formulas in Conjunctive Normal Form (CNF). When a clause becomes empty, its satisfiability is restored by cross-referencing it with previously processed clauses. The method satisfies the first clause by confirming consistency with the remaining clauses. Through bounded transformations, this bidirectional clause interaction preserves clause consistency while avoiding exhaustive search. The approach includes a deterministic strategy for worst-case randomly generated CNF instances and works well for systematically constructed CNF formulas. The existence of a polynomial-time decision method for SAT implies P = NP because SAT is NP-complete.

AUTHOR:

Prakash chandra 1.
Independent Researcher, (pcgango55@gmail.com)
Prof. Lance fortnow *(lfortnow@iit.edu)
Date: January 28, 2026

KEYWORDS

Boolean Satisfiability, SAT, CNF, NP-Complete, P vs NP, Polynomial Time, Clause Cross-Checking.

Previous empty clause P(En), current empty clause C(E1).

BACKGROUND

In theoretical computer science, the Boolean Satisfiability Problem (SAT) holds a fundamental place. SAT determines whether there is an assignment of truth values to variables such that all clauses are satisfied given a Boolean formula expressed in Conjunctive Normal Form (CNF).

The Cook-Levin Theorem, which demonstrated that SAT is NP-complete, established the importance of SAT. This result introduced the concept of NP-completeness by demonstrating that all problems in the class NP can be reduced to SAT in polynomial time. Through polynomial-time reductions from SAT, Karp's later work proved that a variety of combinatorial problems are NP-complete.

Beyond theory, SAT has a wide range of real-world applications in planning, cryptography, software verification, hardware verification, and optimization. The difficulty of solving large SAT instances is closely correlated with the presumed resistance to inversion of cryptographic constructions like SHA-256, which can be encoded as CNF formulas.

There is currently no deterministic polynomial-time algorithm for general SAT, despite significant advancements in SAT solver technology, such as conflict-driven clause learning and heuristic branching. Therefore, the question of whether P = NP is still unanswered.

INTRODUCTION

The canonical NP-complete problem is SAT. All NP-complete problems can be solved in polynomial time if there is a polynomial-time algorithm for SAT.

In contrast to search or backtracking, this paper presents CROSS_IS_SAT, a satisfiability checking technique based on systematic clause cross-checking. The observation that satisfiability is essentially a clause-level property—each clause must maintain at least one consistent literal—serves as the basis for the approach.

There are two main ideas. All subsequent clauses are first cross-checked to remove contradicting literals in order to satisfy the first clause. Second, it is not immediately regarded as proof of unsatisfiability when a clause becomes empty during this process. Rather, the clause is resolved by comparing it to clauses that have already been processed and, if feasible, reestablishing consistency.

Instead of using an exponential exploration of assignments, this method enables the determination of satisfiability through controlled clause transformations. A deterministic

fallback strategy is offered for random or worst-case scenarios, and the method works especially well for systematically constructed

METHODOLOGY

A CNF formula is a set of clauses and literals [see CNF Example 1].

The primary objective of a SAT solver is to find an assignment in polynomial time such that the entire formula evaluates to True (or 1).

Our solver performs this task in the following manner:

Assume we are given a CNF formula F consisting of n variables and m clauses:

F = {C1, C2, C3..... Cm}

Example CNF 1

For example, below is a p cnf 3 8 format, where the formula contains a total of 3 literals

L = {1, 2, 3} (and their negations -L = {-1, -2, -3}, and 8 clauses from (i = 1) to (i = 8):

1 2 3 0

1 2 -3 0

1 -2 3 0

1 -2 -3 0

-1 2 3 0

-1 2 -3 0

-1 -2 3 0

1 -2 -3 0

Our algorithm checks each clause (Ci) step by step, where (i) is the clause index.

The main objective is to ensure that the literal being selected does not conflict with previously selected literals i.e., no literal and its negation (l) and (-l)

are selected together.If a literal does not conflict with previous assignments, it is added to the final set.

After processing, the clauses reduce as follows:

Clause 2: 2 1 3

Clause 3: 3 1

Clause 4: 1

Clause 5: 3 2

Clause 6: 2

Clause 7: 3

Clause 8: 1

Here, Clause (C1) contains literals ({1,2,3}), and all subsequent clauses (C2) to (C8) contain literals from (C1).

Therefore, this example is solved easily without any literal conflicts.

Complexity Analysis Example 1

In this specific example, the algorithm scans each clause only once.

Hence, the time complexity is O(N) (Linear Time).

Since there were no major conflicts among literals, the solver resolves the formula in polynomial time without backtracking.

CNF Example 2: Analysis and Resolution

Now we consider CNF Example 2, where conflicts between literals exist:

```
 p cnf 3 8
-1   2  -3 0
 1  -2   3 0
 1  -2  -3 0
-1   2   3 0
-1  -2   3 0
 1  -2  -3 0
 1   2   3 0
 1   2  -3 0
```

Execution Process for Example 2

Step 1 Initial Assignment:

First, all literals of Clause (C1) are assumed to be True:

C1 = {-1, 2, -3}

Step 2 Conflict Handling in (C2):

When we reach Clause C2 = {1, -2, 3}, we observe that all literals are negations of those in (C1).

Conflict Resolution:

We select a literal (L) from (C2) such that assigning it True and removing its negation from (C1) does not make (C1) empty.

Here, we select literal 1 from (C2).

As a result, its negation (-1) is removed from (C1), but (C1) remains valid with {2, -3}.

Next, literal 1 is selected from Clause (C3), and literal 2 is selected from Clause (C4), and both are assigned True.

After these assignments, no previous clause becomes empty. Hence, these literals are safely added to the final True assignment set.

When moving to Clause (C5), all its literals are negations of previously selected True literals.

In this case, we select literal 3 from (C5).

Its negation (-3) is removed from previous clauses to maintain consistency.

Note:

Assigning (-1) or (-2) as True would have caused previous clauses to become empty, which violates consistency.

Therefore, literal 3 is chosen.

For Clauses (C6) to (C8), none of them contain all negations of previously selected literals.

Thus, previously validated literals remain valid for these clauses.

The final satisfied set is:

Clause 1: 2

Clause 2: 3 1

Clause 3: 1

Clause 4: 3 2

Clause 5: 3

Clause 6: 1

Clause 7: 3 2 1

Clause 8: 2 1

Important Note:

Whenever a literal is assigned True or False, it must be inserted or removed consistently across all clauses where it appears.

This ensures global consistency at every step.

Complexity Analysis Example 2.

Despite conflicts, the solver resolves the formula using Literal Removal and Consistency Checks, without heavy backtracking.

Since each clause is processed using a fixed logic, the complexity remains within Polynomial Time.

CNF Example

1 -2 -3 0

-1 2 -3 0

1 -2 3 0

1 -2 -3 0

-1 2 3 0

-1 -2 3 0

1 2 3 0

1 2 -3 0

To solve this CNF:

. First, all literals of (C1) (1, -2, -3) are set to True.

. Clause (C2) becomes empty since all its literals are negations of (C1).

. We select (-1) from (C2) and remove 1 from (C1).

. From (C3), select (-2); from (C4), select (-2, -3).

. Clause (C5) again becomes empty, so we select 3 and remove its negation from previous clauses.

. Clause C6 = {3, -2, -1}), Clause C7 = {3}.

. Clause (C8) becomes empty.

At this point, assigning any literal from (C8) directly would empty previous clauses.

So, we force-fix literal 1 from (C8) and reinsert it into clauses where it originally appeared (C1, C3, C4, C7), while removing (-1).

This causes (C2) to become empty again.

We then try literals 2 and -3 sequentially.

When 2 is assigned True and its negation is removed, the entire formula becomes satisfied with literals 1, 2, and 3.

Final Assignment (Example 3)

Clause 1: 1

Clause 2: 2

Clause 3: 1 3

Clause 4: 1

Clause 5: 3 2

Clause 6: 3

Clause 7: 3 1 2

Clause 8: 1

Key Observation

If assigning any literal from an empty clause causes previous clauses to become empty, then:

- We fix the literal,
- Form subsets with remaining literals from empty clauses,
- Insert them,
- Remove their negations,

- And attempt to satisfy the formula.

If repeated insertion causes cyclic emptiness, the CNF is declared UNSAT.

Generalized Five-Step Method

1. Make all literals of (C1) valid.
2. In subsequent clauses, remove negations of (C1) literals.
3. If a clause contains only negations of previous literals, select a literal that preserves non-emptiness.
4. If all choices empty previous clauses, form subsets with remaining literals and retry.
5. If conflict persists, move each literal of the empty clause to the first position one by one and then follow steps 1 to 4.

If the CNF remains unsatisfied after applying all steps systematically, it is declared UNSAT.

Experimental Results

Using the above method:

- 128-round SHA-256 CNF generated by CGEN Master was solved in polynomial time.
- 64-round reverse SHA-256 CNF generated by Saeed Natajee was also successfully satisfied in polynomial time.

However, even for unstructured and random CNF, the above 5th steps are expected to work and help in finding a true assignment in polynomial time.

Still, for future work, we propose a method for worst-case random CNF, which can be a potential direction for further research.

Now we explain the method in detail.

Step 1

Assume that a CNF formula is given.

First, we try to satisfy it by applying the 5th steps.

Step 2

Let C1 be the first clause.

We try to make all literals of C1 true.

Step 3

In the subsequent clauses, we remove the negations of the literals of C1 and keep (TRUE) all other literals.

Step 4

If any clause becomes empty, then we apply the previously defined Steps 3, 4, and 5 again.

Step 5

If the CNF is still not satisfied, then we forcefully assign true the first literal of the first empty clause and remove its negation from all clauses.

Let the first empty clause be denoted as C(E1).

Step 6

If assigning the first literal of C(E1) causes the previous clause to become empty, let that clause be denoted P(E1).

Now, we try to fill P(E1) by assigning its literals one by one to true and removing their negations.

Step 7

If, after trying all literals of P(E1), another previous clause P(En) becomes empty, then we apply the same process:

- Try each literal of P(En)one by one,
- Assign it to true,
- Remove its negation,
- And attempt to satisfy the entire CNF.

Step 8.

If the CNF is still not satisfied, then we apply this process to all clauses, i.e.,

from P(E1) to P(En) or C(E1) to C(1) (worst case)

Step 9

If the CNF is still not satisfied, then we repeat the same procedure for each literal of P(E1).

Step 10

If after all these attempts the CNF still cannot be satisfied, then the CNF is declared unsatisfied.

IMPORTANT NOTE:

One important condition must be maintained:

If, after inserting a literal into P(E1) and removing its negation, a previous clause P(En) becomes empty, then that clause must not be made completely empty again.

That is, if filling P(En) and removing its negation again causes P(E1) to become empty, then:

- We must stop further attempts to fill clauses, and
- The CNF must be declared unsatisfied.

In other words, once an empty clause is filled and its negations are removed, at least one literal must remain in that clause.

If this condition is violated, the CNF is considered unsatisfied.

Complexity Analysis:

Example 3 Time Complexity:

Each clause is scanned, and conflicts are resolved locally via subset formation.

Thus, complexity is $O(m \times n)$ (Polynomial Time).

Worst Case (Random CNF):

Worst case occurs when a dependency chain forms from P(E1)) to (P(En).

The algorithm detects circular dependencies.

If filling (P(En)) empties (P(E1)) again, the solver stops and declares UNSAT.

This prevents exponential tree search.

Hence, worst-case complexity remains within O(m^k), where (k) is a small constant.

Optimizations and Clause Reordering.

If the constant k continues to increase after following the steps above, convert the 3-SAT CNF into Square and Pyramid SAT CNF.

The complexity of the Pyramid and Square CNF ranges from a minimum of O(1) to a maximum of O(n). For detailed instructions, please refer to the README.md file.

Theoretical Impact on P vs NP

Based on the methodology and experimental results, this algorithm consistently solves SAT problems in polynomial time for both average and worst cases.

According to the Cook–Levin Theorem, SAT is the first NP-Complete problem.

If SAT can be solved in polynomial time, then all NP problems can be solved in polynomial time.

The way this solver handles complex conflicts (such as SHA-256 CNFs) through Subset Consistency and Recursive Literal Forcing eliminates traditional exponential

backtracking. If this method demonstrates consistent polynomial efficiency across all random and structured CNFs, it may represent a decisive step toward

proving P = NP. This research not only impacts cryptanalysis but also opens a new path toward resolving one of the greatest problems

in Computational Complexity Theory.

CONCLUSION

This paper presented **CROSS_IS_SAT**, a clause cross-checking based method for Boolean satisfiability. By resolving contradictions through structured interaction between clauses, the method avoids exhaustive search. Experimental observations on cryptographic CNF instances demonstrate practical effectiveness. The existence of such a polynomial-time method for SAT implies **P = NP**, carrying fundamental consequences for computational complexity theory.

BIBLIOGRAPHY

1. Cook, S. A., *The Complexity of Theorem-Proving Procedures*, STOC, 1971.
2. Levin, L. A., *Universal Sequential Search Problems*, 1973.
3. Karp, R. M., *Reducibility Among Combinatorial Problems*, 1972.
4. Garey, M. R., Johnson, D. S., *Computers and Intractability*, 1979.
5. Biere et al., *Handbook of Satisfiability*, 2009.
6. Malik, S., Zhang, L., *Boolean Satisfiability: From Theory to Practice*, CACM, 2009.
7. NIST, *Secure Hash Standard (SHA-256)*, FIPS 180-4.

`